TEAM HANDOFF / 01

Password Breach Intelligence and Privacy-Preserving Detection

Implementation summary and demonstration handbook | Prepared 24 October 2026

A teammate guide to what is complete, how the application works, which examples to use, and what the results do and do not prove.

Project scope: A working local Big Data Computing prototype. It combines real breach-hash data, Spark cleaning, compact membership filters, private prefix checking, and synthetic incremental arrivals over real Kafka transport.

What has been completed

Component	Completed implementation
Real dataset	236,495 HIBP SHA-1 records acquired from 120 bounded ranges, with source, acquisition date, selected prefixes and file checksum.
Historical processing	Actual PySpark DataFrames; TXT/CSV/JSON/JSONL input, optional GZIP, validation, normalization, deduplication and clean Parquet output.
Exact storage and indexes	SQLite with an indexed prefix column, plus genuine packed Bloom and bucket-based Cuckoo Filters.
Private checker	Browser-local SHA-1; only a five-character prefix is sent; exact suffix comparison occurs locally.
Incremental updates	Kafka KRaft broker, Spark Structured Streaming, checkpoint recovery and idempotent backend batch ingestion.
Dashboard and validation	Four working sections, persisted jobs, real metrics/charts, measured benchmarks, automated tests and an isolated reset workflow.

The two main workflows



Stack: React + TypeScript + Vite, Tailwind CSS, Recharts, FastAPI/Pydantic, SQLite, PySpark and Apache Kafka.

Essential distinction: The private checker retrieves candidates from SQLite. Bloom and Cuckoo Filters are maintained alongside the database for full-hash membership experiments; they cannot enumerate suffixes for a prefix.

TEAM HANDOFF / 02

What happens when a password is entered

The password-checking path, using a fixed public example.

1. Enter a public demo string. Type **password**. Typing alone does not issue a password-range query; the user must click **Check exposure**.

2. Hash inside the browser. Web Crypto computes SHA-1 from the exact UTF-8 input. Password case and spaces are preserved.

```
password
SHA-1: 5BAA61E4C9B93F3F0682250B6CF8331B7EE68FD8
Prefix: 5BAA6
Suffix: 1E4C9B93F3F0682250B6CF8331B7EE68FD8
```

3. Send only the prefix. The browser calls our local backend. It does not send the plaintext, complete client hash or client suffix.

```
GET /api/range/5BAA6
```

4. Retrieve candidates. SQLite returns suffixes and occurrence counts for loaded hashes beginning with that prefix. Our acquired 5BAA6 range contained 1,978 candidates.

5. Compare locally. The browser compares its suffix with the returned candidates, displays found/not found in the loaded dataset, and clears the input after success.

Why most other passwords return "not found"

The sample contains only **120 of 1,048,576 possible prefixes**, approximately **0.0114%**. The range for **password** was deliberately included as a reliable real-data example. A common password can still be outside our selected ranges.

Public example	Observed result	Interpretation
password	Found	Real HIBP sample match.
123456 / qwerty / admin / password123	Not found	Their prefixes are outside the selected sample ranges.
public-synthetic-control-0	Found	Synthetic cleaning-fixture control.
public-synthetic-control-1	Found	Synthetic cleaning-fixture control.

Correct interpretation: "Not found in our loaded dataset" is not a guarantee of safety. Queries search the locally loaded sample; they do not perform a fresh global HIBP search. Five-character prefix queries do not guarantee a fixed anonymity-set size.

SHA-1 is used to match this corpus, not as a recommendation for account-password storage. No plaintext recovery or password cracking is performed.

TEAM HANDOFF / 03

Cleaning, duplicates and the second dataset

What Spark does, why counts change, and what is already loaded.

Historical cleaning pipeline

Spark reads the selected input, trims surrounding hash whitespace, converts hexadecimal letters to uppercase, validates full 40-character SHA-1 hashes and non-negative integer counts, rejects invalid rows, and deduplicates by normalized full hash. Internal malformed characters are not silently removed.

For duplicate hashes with different counts, the policy is **maximum observed count**, not a sum of duplicated copies. Source labels are retained. Clean output is written to Parquet and then merged into SQLite and the filters.

The cleaning fixture has already been processed

The separate **cleaning_test.csv** has canonical rows plus exact duplicates, lowercase copies, surrounding spaces/tabs, empty hashes, invalid characters, wrong lengths and invalid counts. It is a synthetic data-quality exercise, not additional verified breach evidence.

Metric	Real sample	Fixture only	Sample + fixture
Input records	236,495	21	236,516
Valid before deduplication	236,495	14	236,509
Invalid records	0	7	7
Duplicate valid rows removed	0	9	12
Unique valid hashes	236,495	5	236,497
Rows normalized	0	7	7

Fixture rejection breakdown: 2 missing hashes, 3 invalid hashes and 2 invalid counts. Each rejected row has one primary reason. Normalized rows overlap validity categories and must not be added to the other totals.

Why the fixture adds only two new hashes

Its five unique valid hashes include **three selected from the real sample** and **two distinct synthetic controls**. Those three already exist in the main corpus. The initial fixture import therefore introduced only two additional exact records.

Repeated imports are expected to add zero new records. Cleaning metrics describe the selected input file. Database ingestion metrics describe whether those cleaned hashes already exist in the loaded store. Reprocessing the fixture still removes nine duplicate input rows even when it inserts zero new database rows.

Combined cleaning removes 12 duplicates rather than 9 because the three selected real hashes overlap between the sample and fixture. Total loaded records may exceed the real-sample count because explicitly labelled test and stream records are also retained.

TEAM HANDOFF / 04

Bloom and Cuckoo Filters

Their role, implementation details and actual measured comparison.

Feature	Bloom Filter	Cuckoo Filter
Stores	Packed bits. Several calculated positions represent each inserted hash.	Short fingerprints in buckets. Each fingerprint has two candidate bucket locations.
Membership answer	Any required zero bit means definitely absent; all bits set means possibly present.	A matching fingerprint means possibly present; no match means absent when the filter is complete.
Deletion	Unsupported: clearing shared bits can damage other memberships.	Supported for a known inserted item. The demonstration uses a separate synthetic index.
Our implementation	Deterministic BLAKE2b positions; standard Bloom sizing; target 1% FPR at provisioned capacity.	Four slots per bucket, 10-bit fingerprints, up to 500 relocations, rollback on failed insertion.

A false positive means an absent item appears possibly present because bits or fingerprints collide. A positive filter answer alone is not exact proof. SQLite remains authoritative. On capacity/insertion failure, exact records are preserved and a larger filter can be rebuilt.

Saved benchmark: 236,499 unique loaded hashes

This snapshot includes the real sample plus labelled test records. Both filters received the same data and identical query workloads. Measurements used one warmup and five timed batches; per-query latency is derived from repeated batch timing.

Measured value	Bloom	Cuckoo
Packed index bytes	566,715 (about 553 KiB)	2,097,152 (2 MiB)
Build time	1,799.75 ms	896.01 ms
Median lookup latency	5.88 microseconds	4.28 microseconds
Observed false positives	4 / 10,000 absent queries	12 / 10,000 absent queries
Inserted member misses	0	0
Failed insertions	0	0

Observed outcome: Cuckoo was faster and Bloom used less packed storage in this configuration. Their FPR settings/load differ; this is not equal-FPR tuning or proof that one is universally better. Sizes exclude SQLite, Spark and the Python runtime.

Measured scales were 10K, 100K and 236,499 hashes. All inserted members were checked. The controlled 100-item Cuckoo deletion succeeded with zero other-member misses; fingerprint collisions can still leave a deleted item matching.

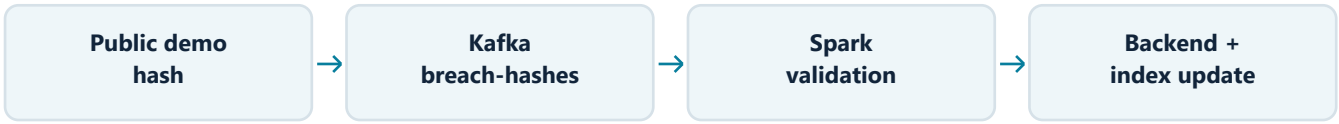
Presentation wording: "We experimentally compare full-hash membership structures. Our privacy-enabled checker uses SQLite prefix retrieval and local exact comparison." Do not claim the Bloom Filter lists all suffixes for a prefix.

TEAM HANDOFF / 05

Kafka, Spark and incremental updates

Real transport and processing, with explicitly simulated breach arrivals.

What is live? Kafka and Spark process real messages. **What is simulated?** The new breach arrival. There is no automatic subscription to newly discovered real HIBP breaches in this prototype.



1. Start simulation. The backend chooses a deterministic public demo string absent from the current store and delays its event by ten seconds.

2. Produce one event. The producer sends JSON with event ID, full synthetic SHA-1, count, source and emission time to Kafka. Its source is synthetic-stream-demo.

3. Process a micro-batch. Spark Structured Streaming consumes Kafka, applies validation/normalization and sends a cleaned batch to the internal ingestion endpoint. The trigger is about five seconds; actual latency can be longer.

4. Publish the update. The backend deduplicates against SQLite, updates the prefix index and both filters, and advances the index version. Spark checkpoints allow recovery; repeated batch IDs/checksums do not duplicate insertion.

Latest recorded before/after proof

Evidence	Before	After
ClassDemoOnly-2026!-3	Not found	Found
Total loaded hashes	236,501	236,502
Index version	55	56

One Kafka event was produced; Spark was running and applied batch **5**. These are recorded test observations, not a promise that background processes will stay running after a terminal closes.

How to demonstrate it reliably

Open two dashboard tabs: **Live Stream** in one and **Password Checker** in the other. Start simulation, copy the newly displayed public string, and check it within the ten-second delay. It should be absent. Wait for an applied batch and increased index version, then check the same string again: it should be found.

Use a new displayed string on every run. The example above is already loaded. If the initial ten-second window is missed, run another simulation rather than deleting real corpus records.

The producer stopping after one event is normal. Spark should remain running, waiting for later messages. Kafka TCP reachability alone is insufficient proof: also show the applied batch and changed lookup result.

TEAM HANDOFF / 06

Show that every section works

An 8-10 minute walkthrough with observable evidence.

Dashboard section	Action	Evidence to show
Password Checker 1 minute	Check password, then public-synthetic-control-0. Explain real vs synthetic sources.	Both are found. An unrepresented common password can be not found because coverage is limited.
Privacy 1 minute	Open browser Developer Tools > Network. Filter /range/ and check password.	GET /api/range/5BAA6, with no request body containing plaintext, complete hash or client suffix.
Data Pipeline: real 1 minute	Click Process real sample; wait for the completed job.	236,495 input/unique; zero invalid and cleaning duplicates for this acquired sample.
Data Pipeline: fixture 1 minute	Click Process cleaning fixture. Show the fixture file and manifest.	21 input, 7 invalid, 9 duplicate valid rows removed, 7 normalized and 5 unique.
Combined data 1 minute	Click Process sample + fixture. Explain the overlap.	236,516 input, 7 invalid, 12 duplicates removed and 236,497 unique.
Live Stream 2 minutes	Use two tabs for the absent -> found demonstration from page 5.	Producer acknowledgment, applied Spark batch, one new exact record and increased index version.
Filter Comparison 1-2 minutes	Run measured comparison and wait for its background job.	Actual sizes/timings, member misses = 0, failed inserts = 0 and false-positive counts with denominators.
Cuckoo deletion 30 seconds	Show the controlled deletion result.	Deleted = true, other-member misses = 0. Genuine breach records are not removed.

Keep the evidence consistent

Top-level loaded totals count all stored unique hashes. A processing report counts the selected files. Benchmark results describe the snapshot when that benchmark ran. Those numbers can legitimately differ after additional synthetic arrivals or repeated imports.

"Zero newly inserted" on a repeated import is evidence of exact-store deduplication, not a failed cleaning job. "Zero observed false positives in N tests", if it occurs in a future run, is not zero probability.

Automated proof commands in the prepared WSL environment

```
wsl -d BDC-Spark -- bash -c 'source /mnt/d/BDC/scripts/wsl_env.sh;
python scripts/healthcheck.py'
```

```
wsl -d BDC-Spark -- bash -c 'source /mnt/d/BDC/scripts/wsl_env.sh;
python scripts/check_stream_demo.py'
```

The line breaks above are for readability: enter each wsl command as a single line. The stream proof saves artifacts/metrics/latest-stream-proof.json and uses a public generated string. It does not bypass Kafka by inserting directly into the database.

TEAM HANDOFF / 07

Team handoff and references

Files to know, startup instructions, validation and remaining limits.

Restart the already prepared machine

From PowerShell in **D:\BDC**, run this in the first terminal and **keep it open** for Spark:

```
wsl -d BDC-Spark -- bash /mnt/d/BDC/scripts/start_wsl_stack.sh
```

In a second PowerShell terminal at D:\BDC:

```
cd frontend
npm run dev
```

Use the URL Vite prints. Normal port: 5173; inspected session: 5174. API: 8000. WSL Kafka: 9094. The local development admin token is local-demo-admin unless changed. These localhost addresses work only on the machine running the project.

Implementation map

Topic	Repository file
Browser hashing and local match	frontend/src/privacy.ts
Bloom + Cuckoo implementations	backend/app/filters.py
Exact storage and prefix retrieval	backend/app/store.py
Historical Spark cleaning	spark/batch_pipeline.py
Kafka + Spark micro-batches	spark/streaming_pipeline.py
APIs, jobs and producer controls	backend/app/main.py
Benchmarks / fixture generator	backend/app/benchmark.py / scripts/create_cleaning_fixture.py

Validation completed: 22 backend/Spark regression tests, 2 real Kafka/Spark integration tests and 2 frontend privacy tests passed. The 10-test component rerun overlaps the regression suite and is not an additional independent test count. TypeScript/Vite build and Compose configuration checks passed.

Runtime limits: Verified on isolated Ubuntu WSL2 with Python 3.10.12, Java 17.0.20.1, Spark 3.5.7 local[2] and Kafka 3.7.1. Docker image execution remains unverified because Docker Desktop failed to start its engine. A million-record run, multi-node scalability and production deployment hardening were not measured.

Storage note: Active WSL SQLite is /opt/bdc-state/breach.sqlite inside the isolated runtime. Avoid simultaneous Windows/Linux access to a live SQLite WAL database. The prepared WSL distribution and runtime assets are local setup, not part of a source-only checkout. Use README.md and docs/LOCAL_WSL.md for fresh-machine setup.

References for presentation slides

1. Bloom, B. H. (1970). Space/Time Trade-offs in Hash Coding with Allowable Errors. Communications of the ACM. ACM paper.
2. Fan, B., Andersen, D. G., Kaminsky, M., and Mitzenmacher, M. D. (2014). Cuckoo Filter: Practically Better Than Bloom. CoNEXT. Authors' publication page.
3. HIBP. Pwned Passwords range-query documentation.
4. Apache Spark 3.5.7. Structured Streaming + Kafka integration.